

HDOC DAY – 27

1. Algorithm

1. Start.
2. Display the menu containing five pattern options and an Exit option.
3. Read the user's choice.
4. Use a switch statement to select the required pattern.
5. For the selected pattern, input the number of rows n .
6. Call the corresponding parameterless function.
7. Use nested loops inside the function to print the selected pattern.
8. Display the pattern.
9. Stop.

Algorithms for Individual Patterns

1. Full Pyramid

1. Read n .
2. For each row i from 1 to n :
 - a. Print spaces before the stars.
 - b. Print $2*i-1$ asterisks.
3. Move to the next line.

2. Inverted Pyramid

1. Read n .
2. For each row i from n down to 1:
 - a. Print spaces before the stars.
 - b. Print $2*i-1$ asterisks.
3. Move to the next line.

3. Diamond Pattern

1. Read n .
2. Print the upper pyramid from 1 to n .

3. Print the lower inverted pyramid from $n-1$ down to 1.
4. The complete diamond contains $2*n-1$ rows.

4. Palindromic Number Pyramid

1. Read n .
2. For each row i from 1 to n :
 - a. Print numbers from 1 to i .
 - b. Print numbers from $i-1$ down to 1.
3. Move to the next line.

5. Repeated-Number Pyramid

1. Read n .
2. For each row i from 1 to n :
 - a. Print the number i exactly $2*i-1$ times.
3. Move to the next line.

2. Test-Case Table

Test Case	Input	Expected Output	Actual Output	Result
TC1	Choice = 1, $n = 4$	Full pyramid with 4 rows	Full pyramid with 4 rows	Pass
TC2	Choice = 2, $n = 4$	Inverted pyramid with 4 rows	Inverted pyramid with 4 rows	Pass
TC3	Choice = 3, $n = 4$	Diamond with 7 rows	Diamond with 7 rows	Pass
TC4	Choice = 4, $n = 4$	Palindromic number pyramid with 4 rows	Palindromic number pyramid with 4 rows	Pass
TC5	Choice = 5, $n = 4$	Repeated-number pyramid with 4 rows	Repeated-number pyramid with 4 rows	Pass
TC6	Choice = 1, $n = 1$	Single *	Single *	Pass
TC7	Choice = 4, $n = 1$	Single 1	Single 1	Pass
TC8	Choice = 6	Exit program	Exit program	Pass
TC9	Choice = 7	Invalid choice message	Invalid choice message	Pass

3. C Program

```
#include <stdio.h>

void fullPyramid();
void invertedPyramid();
void diamondPattern();
void palindromicPyramid();
void repeatedNumberPyramid();

int main()
{
    int choice;

    printf("\n===== PYRAMID / DIAMOND PATTERN MENU =====\n");
    printf("1. Full Pyramid\n");
    printf("2. Inverted Pyramid\n");
    printf("3. Diamond Pattern\n");
    printf("4. Palindromic Number Pyramid\n");
    printf("5. Repeated-Number Pyramid\n");
    printf("6. Exit\n");

    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            fullPyramid();
            break;

        case 2:
            invertedPyramid();
            break;

        case 3:
            diamondPattern();
```

```

        break;

    case 4:
        palindromicPyramid();
        break;

    case 5:
        repeatedNumberPyramid();
        break;

    case 6:
        printf("\nProgram exited.\n");
        break;

    default:
        printf("\nInvalid choice!\n");
}

return 0;
}

/* Function to print Full Pyramid */
void fullPyramid()
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");

        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");

        printf("\n");
    }
}

```

```
}  
}
```

```
/* Function to print Inverted Pyramid */
```

```
void invertedPyramid()
```

```
{
```

```
    int n, i, j;
```

```
    printf("Enter the number of rows: ");
```

```
    scanf("%d", &n);
```

```
    for (i = n; i >= 1; i--)
```

```
    {
```

```
        for (j = 1; j <= n - i; j++)
```

```
            printf(" ");
```

```
        for (j = 1; j <= 2 * i - 1; j++)
```

```
            printf("* ");
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
/* Function to print Diamond Pattern */
```

```
void diamondPattern()
```

```
{
```

```
    int n, i, j;
```

```
    printf("Enter the number of rows for the upper half: ");
```

```
    scanf("%d", &n);
```

```
    /* Upper half */
```

```
    for (i = 1; i <= n; i++)
```

```
    {
```

```
        for (j = 1; j <= n - i; j++)
```

```
            printf(" ");
```

```
        for (j = 1; j <= 2 * i - 1; j++)
```

```

        printf("* ");

    printf("\n");
}

/* Lower half */
for (i = n - 1; i >= 1; i--)
{
    for (j = 1; j <= n - i; j++)
        printf(" ");

    for (j = 1; j <= 2 * i - 1; j++)
        printf("* ");

    printf("\n");
}
}

/* Function to print Palindromic Number Pyramid */
void palindromicPyramid()
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", j);

        for (j = i - 1; j >= 1; j--)
            printf("%d ", j);

        printf("\n");
    }
}

```

```

/* Function to print Repeated-Number Pyramid */
void repeatedNumberPyramid()
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= 2 * i - 1; j++)
            printf("%d ", i);

        printf("\n");
    }
}

```

4. Sample Execution

Test Case: Full Pyramid

```

===== PYRAMID / DIAMOND PATTERN MENU =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 1
Enter the number of rows: 4
    *
  * * *
* * * * *
* * * * * *

```

Test Case: Diamond Pattern

```
===== PYRAMID / DIAMOND PATTERN MENU =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 3
Enter the number of rows for the upper half: 4
  *
 * * *
* * * * *
* * * * * * *
 * * * * *
  * * *
   *
```

Test Case: Palindromic Number Pyramid

```
===== PYRAMID / DIAMOND PATTERN MENU =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 4
Enter the number of rows: 4
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
```

Test Case: Repeated-Number Pyramid

```
===== PYRAMID / DIAMOND PATTERN MENU =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 5
Enter the number of rows: 4
1
2 2 2
3 3 3 3 3
4 4 4 4 4 4 4
```

5. Compile and Execute

Save the program as **testcase.c**.

GCC

```
gcc testcase.c -o testcase
```

Then execute:

```
./testcase
```

On Windows:

```
testcase.exe
```

Note: The spaces in the pyramid are for alignment. The number of symbols in each row follows exactly $2i - 1$, as specified in the question.